1.Create a React Timer component that can start, pause, and reset. Display the time in mm:ss format using React hooks (useState, useEffect).

```
import React, { useState, useEffect } from "react";


function App() {
 const [time, setTime] = useState(0); // time in seconds
 const [running, setRunning] = useState(false);


 useEffect(() => {
  let timer;
  if (running) {
   timer = setInterval(() => setTime((t) => t + 1), 1000);
  }
  return () => clearInterval(timer);
 }, [running]);


 const format = (t) => {
  const m = String(Math.floor(t / 60)).padStart(2, "0");
  const s = String(t % 60).padStart(2, "0");
  return `${m}:${s}`;
 };


 return (
  <div style={{ textAlign: "center", marginTop: "50px" }}>
   <h3>React Timer</h3>
   <h2>{format(time)}</h2>
   <button onClick={() => setRunning(true)}>Start</button>
   <button onClick={() => setRunning(false)}>Pause</button>
```

```
      <button onClick={() => { setTime(0); setRunning(false); }}>Reset</button>
    </div>
  );
}

export default App;
```

2. Create a React Counter component with Increment, Decrement, and Reset buttons. Maintain the count in state.

```
import React, { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h3>React Counter</h3>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

export default App;
```

3. Create a React Counter that stores count in localStorage so that the value persists after refresh.

```
import React, { useState, useEffect } from "react";


function App() {
  const [count, setCount] = useState(() => {
    return Number(localStorage.getItem("count")) || 0;
  });


  useEffect(() => {
    localStorage.setItem("count", count);
  }, [count]);


  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h3>Persistent Counter</h3>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}


export default App;
```

1. **4. Build a React Timer component with an additional Lap button to record lap times.**

```
import React, { useState, useEffect } from "react";

function App() {
  const [time, setTime] = useState(0);
  const [running, setRunning] = useState(false);
  const [laps, setLaps] = useState([]);

  useEffect(() => {
    let timer;
    if (running) timer = setInterval(() => setTime(t => t + 1), 1000);
    return () => clearInterval(timer);
  }, [running]);

  const format = (t) => {
    const m = String(Math.floor(t / 60)).padStart(2, "0");
    const s = String(t % 60).padStart(2, "0");
    return `${m}:${s}`;
  };

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h3>React Timer with Laps</h3>
      <h2>{format(time)}</h2>
      <button onClick={() => setRunning(true)}>Start</button>
```

```
    <button onClick={() => setRunning(false)}>Pause</button>
    <button onClick={() => { setTime(0); setRunning(false); setLaps([]); }}>Reset</button>
    <button onClick={() => setLaps([...laps, format(time)])}>Lap</button>

    <div style={{ marginTop: "20px" }}>
     {laps.map((lap, i) => (
       <p key={i}>Lap {i + 1}: {lap}</p>
     ))}
    </div>
   </div>
  );
}

export default App;
```

15. Create an Express Server with a simple route /hello that returns { "message": "Hello MERN Stack" }.

```
const express = require("express");

const app = express();

const PORT = 3000;


app.get("/hello", (req, res) => {

  res.json({ message: "Hello MERN Stack" });

});


app.listen(PORT, () => {

  console.log(` Server running on http://localhost:${PORT}` );

});
```

12. Perform MongoDB CRUD Operations (using Node.js or shell): Insert multiple student records, Retrieve all students, Filter by branch, Update one student's cgpa, Delete one student record.

✅ **Step 1: Create / Switch to Database**

```
use college;
```

✅ **Step 2: Insert Multiple Student Records**

```
db.students.insertMany([
 { name: "Mani", branch: "CSE", cgpa: 8.9 },
 { name: "Arun", branch: "ECE", cgpa: 8.5 },
 { name: "Divya", branch: "CSE", cgpa: 9.2 },
 { name: "Priya", branch: "IT", cgpa: 7.8 }
]);
```

✔ This creates a collection named **students** and inserts **4 documents**.

✅ **Step 3: Retrieve All Students**

```
db.students.find();
```

✔ Displays all student records stored in the collection.

✅ **Step 4: Filter Students by Branch**

```
db.students.find({ branch: "CSE" });
```

✔ Shows only students from the **CSE** branch.

✅ **Step 5: Update One Student's CGPA**

```
db.students.updateOne(
 { name: "Priya" },
 { $set: { cgpa: 8.2 } }
);
```

✔ Updates **Priya's CGPA** to **8.2**.

✅ Step 6: Delete One Student Record

```
db.students.deleteOne({ name: "Arun" });
```

✔ Deletes the student named **Arun**.

✅ Step 7: Verify Changes

```
db.students.find();
```

✔ Displays the updated list of students after insertion, update, and deletion operations.

7. Build an Express application with routes for managing courses: GET /api/courses, POST /api/courses, DELETE /api/courses/:id. Use mock JSON data (no database) and test in Postman.

```
// app.js
const express = require("express");
const app = express();
app.use(express.json()); // To parse JSON data from requests

// Mock data (acts like a temporary database)
let courses = [
 { id: 1, name: "Web Development" },
 { id: 2, name: "Data Structures" },
 { id: 3, name: "Database Systems" }
];

// ✅ GET /api/courses – Retrieve all courses
app.get("/api/courses", (req, res) => {
 res.json(courses);
});

// ✅ POST /api/courses – Add a new course
app.post("/api/courses", (req, res) => {
 const newCourse = {
  id: courses.length + 1,
  name: req.body.name
 };
 courses.push(newCourse);
 res.status(201).json(newCourse);
```

```
});

// ✅ DELETE /api/courses/:id – Delete a course by ID
app.delete("/api/courses/:id", (req, res) => {
  const id = parseInt(req.params.id);
  courses = courses.filter(course => course.id !== id);
  res.json({ message: `Course with id ${id} deleted` });
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
```

8, Create an Express server with the following CRUD routes for students: GET /api/students, POST /api/students, PUT /api/students/:id, DELETE /api/students/:id. Use mock data (in-memory array) and display JSON responses.

```
const express = require("express");
const app = express();
const port = 3000;

app.use(express.json()); // Middleware to parse JSON

// Mock in-memory student data
let students = [
 { id: 1, name: "Mani", branch: "CSE" },
 { id: 2, name: "Arun", branch: "ECE" },
 { id: 3, name: "Divya", branch: "IT" }
];

// ✅ GET: Retrieve all students
app.get("/api/students", (req, res) => {
 res.json(students);
});

// ✅ POST: Add a new student
app.post("/api/students", (req, res) => {
 const newStudent = {
  id: students.length + 1,
  name: req.body.name,
  branch: req.body.branch
 };
```

```
  students.push(newStudent);
  res.status(201).json(newStudent);
});

// ✅ PUT: Update a student by ID
app.put("/api/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const student = students.find(s => s.id === id);

  if (!student) {
    return res.status(404).json({ message: "Student not found" });
  }

  student.name = req.body.name || student.name;
  student.branch = req.body.branch || student.branch;
  res.json({ message: "Student updated successfully", student });
});

// ✅ DELETE: Remove a student by ID
app.delete("/api/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  students = students.filter(s => s.id !== id);
  res.json({ message: `Student with id ${id} deleted successfully` });
});
// ✅ Start the server
app.listen(port, () => {
  console.log(`Server running on http://localhost:${port}`);
});
```

9. Build an Express REST API using Node.js + MongoDB (Mongoose) to perform full CRUD operations on student data: POST /students, GET /students, PUT /students/:id, DELETE /students/:id. Test all routes using Postman.

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();

app.use(express.json()); // Middleware to parse JSON

// ✅ Connect to MongoDB (Change URI if needed)
mongoose.connect("mongodb://127.0.0.1:27017/studentdb", {
 useNewUrlParser: true,
 useUnifiedTopology: true
})
.then(() => console.log("✅ Connected to MongoDB"))
.catch(err => console.error("❌ MongoDB connection error:", err));

// ✅ Define Schema and Model
const studentSchema = new mongoose.Schema({
 name: String,
 branch: String,
 cgpa: Number
});
const Student = mongoose.model("Student", studentSchema);

// ✅ POST: Create new student
app.post("/students", async (req, res) => {
 try {
```

```
    const student = new Student(req.body);
    await student.save();
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ✅ GET: Retrieve all students
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// ✅ PUT: Update a student by ID
app.put("/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(req.params.id, req.body, {
      new: true
    });
    if (!student) return res.status(404).json({ message: "Student not found" });
    res.json({ message: "Student updated successfully", student });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ✅ DELETE: Remove a student by ID
```

```
app.delete("/students/:id", async (req, res) => {
 try {
  const student = await Student.findByIdAndDelete(req.params.id);
  if (!student) return res.status(404).json({ message: "Student not found" });
  res.json({ message: "Student deleted successfully" });
 } catch (err) {
  res.status(400).json({ error: err.message });
 }
});

// ✅ Start the Server
const PORT = 3000;
app.listen(PORT, () => console.log(` 🚀 Server running on http://localhost:${PORT}` ));
```

10. Develop REST API routes for student data with filtering and pagination: POST /students – Add a new student, GET /students?branch=&page=&limit= – Retrieve students with optional filtering and pagination. Use Express + MongoDB and test with Postman.

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();

app.use(express.json());

// ✅ Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/studentdb", {
 useNewUrlParser: true,
 useUnifiedTopology: true
})
.then(() => console.log("✅ Connected to MongoDB"))
.catch(err => console.error("❌ Connection error:", err));

// ✅ Define Schema and Model
const studentSchema = new mongoose.Schema({
 name: String,
 branch: String,
 cgpa: Number
});
const Student = mongoose.model("Student", studentSchema);

// ✅ POST /students – Add a new student
app.post("/students", async (req, res) => {
```

```
 try {
   const student = new Student(req.body);
   await student.save();
   res.status(201).json(student);
 } catch (err) {
   res.status(400).json({ error: err.message });
 }
});

// ✅ GET /students?branch=&page=&limit= – Retrieve with filter + pagination
app.get("/students", async (req, res) => {
 try {
   const { branch, page = 1, limit = 3 } = req.query; // default: page=1, limit=3
   const filter = branch ? { branch: branch } : {};

   const students = await Student.find(filter)
    .skip((page - 1) * limit)  // skip previous pages
    .limit(Number(limit));     // limit records per page

   const total = await Student.countDocuments(filter);

   res.json({
    total,
    page: Number(page),
    pages: Math.ceil(total / limit),
    students
   });
 } catch (err) {
```

```
    res.status(500).json({ error: err.message });
  }
});

// ✅ Start the server
const PORT = 3000;
app.listen(PORT, () => console.log(` 🚀 Server running on http://localhost:${PORT}` ));
```

6. Create a React Password Strength Meter that includes a visual progress bar and indicators for each rule (length, uppercase, number, special character).

```
import React, { useState } from 'react';

export default function PasswordStrengthChecker() {
  const [password, setPassword] = useState('');
  const [strength, setStrength] = useState({ level: '', score: 0, color: '' });

  const checkStrength = (pwd) => {
    let score = 0;
    if (pwd.length >= 8) score += 25;
    if (/[A-Z]/.test(pwd)) score += 25;
    if (/[0-9]/.test(pwd)) score += 25;
    if (/[^A-Za-z0-9]/.test(pwd)) score += 25;

    let level = '', color = '';
    if (score <= 25) { level = 'Weak'; color = 'red'; }
    else if (score <= 50) { level = 'Fair'; color = 'orange'; }
    else if (score <= 75) { level = 'Good'; color = 'blue'; }
    else { level = 'Strong'; color = 'green'; }

    setStrength({ level, score, color });
  };

  const handleChange = (e) => {
    const newPwd = e.target.value;
    setPassword(newPwd);
    checkStrength(newPwd);
```

```
};

// ✅ Simplified UI (no box, no shadow)
return (
  <div style={{
    display: 'flex', flexDirection: 'column',
    alignItems: 'center', justifyContent: 'center',
    height: '100vh', backgroundColor: '#f5f5f5'
  }}>
    <h2>Password Strength Checker</h2>
    <input
      type="password"
      placeholder="Enter your password"
      value={password}
      onChange={handleChange}
      style={{ width: '250px', padding: '8px', margin: '10px 0', borderRadius: '5px', border: '1px solid #ccc' }}
    />

    <div style={{ width: '250px', height: '10px', backgroundColor: '#eee', borderRadius: '5px', marginBottom: '10px' }}>
      <div style={{
        width: `${strength.score}%`, height: '10px',
        backgroundColor: strength.color, borderRadius: '5px'
      }}></div>
    </div>

    {password && (
```

```
        <p style={{ color: strength.color, fontWeight: 'bold' }}>
          {strength.level} Password
        </p>
      )}
    </div>
  );
}
```

11. Implement partial update (PATCH) and delete functionality for student data using Express and MongoDB: PATCH /students/:id – Update selected fields, DELETE /students/:id – Remove a record. Handle 404 and validation errors properly.

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();

app.use(express.json());

// ✅ Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/studentdb", {
 useNewUrlParser: true,
 useUnifiedTopology: true
})
.then(() => console.log("✅ MongoDB connected"))
.catch(err => console.error("❌ Connection error:", err));

// ✅ Schema & Model
const studentSchema = new mongoose.Schema({
 name: String,
 branch: String,
 cgpa: Number
});
const Student = mongoose.model("Student", studentSchema);

// ✅ PATCH: Partial Update Student by ID
app.patch("/students/:id", async (req, res) => {
 try {
```

```
    const student = await Student.findByIdAndUpdate(
      req.params.id,
      { $set: req.body }, // Only update given fields
      { new: true, runValidators: true }
    );

    if (!student) {
      return res.status(404).json({ message: "Student not found" });
    }

    res.status(200).json({ message: "Student partially updated", student });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ✅ DELETE: Remove a student by ID
app.delete("/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) {
      return res.status(404).json({ message: "Student not found" });
    }
    res.status(200).json({ message: "Student deleted successfully" });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
```

```
// ✅ Start Server
const PORT = 3000;
app.listen(PORT, () => console.log(` 🚀 Server running on http://localhost:${PORT}` ));
```

5.Develop a Password Strength Checker in React that evaluates password strength based on: minimum 8 characters, at least one uppercase letter, one number, and one special character. Display result as Weak / Medium / Strong with color indication.

```
import React, { useState } from "react";


function App() {
 const [password, setPassword] = useState("");
 const [strength, setStrength] = useState("");


 const checkStrength = (pwd) => {
  if (pwd === "") return "";
  if (pwd.length < 8) return "Weak";
  if (/[A-Z]/.test(pwd) && /[0-9]/.test(pwd) && /[^A-Za-z0-9]/.test(pwd))
   return "Strong";
  return "Medium";
 };


 const getColor = (level) => {
  if (level === "Weak") return "red";
  if (level === "Medium") return "orange";
  if (level === "Strong") return "green";
  return "black";
 };


 const handleChange = (e) => {
  const value = e.target.value;
  setPassword(value);
```

```
    setStrength(checkStrength(value));
  };

  return (
    <div style={{ textAlign: "center", marginTop: "100px" }}>
      <h2>Password Strength Checker</h2>
      <input
        type="password"
        value={password}
        onChange={handleChange}
        placeholder="Enter your password"
        style={{ padding: "8px", width: "250px" }}
      />
      <p style={{ color: getColor(strength), fontWeight: "bold" }}>
        {strength && `${strength} Password`}
      </p>
    </div>
  );
}

export default App;
```